ACTIVITAT AVALUABLE AC4
Mòdul: M0612- Desenvolupament web en entorn client **U1 – Fonaments de JavaScript** **Professor:** Albert Guardiola **Data límit d'entrega:** 22/10/2026 23:59 **Mètode d'entrega: App publicada a Github Pages (URL del repo)** **Avaluació de la tasca: La nota serà la del control posterior que es realitzarà a classe (50%) més la comprovació de la funcionalitat desplegada a GitHub Pages (50%).**

Resultats de l'aprenentatge:
RA1, RA2, RA3, RA4

AC4- Dashboard d'Inventari

L'objectiu d'aquesta pràctica és construir la lògica d'un dashboard interactiu. Treballaràs amb un array d'objectes anomenat productes i aprendràs a manipular-lo combinant els mètodes clàssics de JavaScript amb la **programació funcional** i les noves funcions d'**immutabilitat d'ES2023**.

Tasques Pas a Pas

Tasca 1: Mètriques del Dashboard (Programació Funcional i Accés)

Al dashboard tenim 4 targetes superiors. Has de calcular els seus valors a la funció actualitzarMetriques():

1.1 Total de l'inventari amb reduce

El mètode reduce acumula tots els elements d'un array en un únic valor final.

- **Sintaxi:** array.reduce((acumulador, element) => nouAcumulador, valorInicial)
- **Exemple de sintaxi:**

```
const suma = nombres.reduce((acc, num) => acc + num, 0);
```

1.2 Accés a l'últim element amb at()

El mètode at() permet accedir als elements d'un array fent servir índexs positius o negatius. L'índex -1 fa referència a l'últim element.

- **Sintaxi:** array.at(indexNegatiu)
- **Exemple de sintaxi:**

```
const ultimElement = llista.at(-1);
```

1.3 Comprovar si hi ha faltes de stock amb some

El mètode some retorna true si **almenys un** element de l'array compleix la condició.

- **Sintaxi:** array.some(element => condicio)
- **Exemple de sintaxi:**

```
const hiHaNegatius = nombres.some(n => n < 0);
```

1.4 Validar preus vàlids amb every

El mètode every retorna true només si **tots** els elements compleixen la condició.

- **Sintaxi:** array.every(element => condicio)
- **Exemple de sintaxi:**

```
const totsPositius = nombres.every(n => n > 0);
```

Tasca 2: Afegir i Editar Productes (push, with i generació d'ID)

A l'esdeveniment submit del formulari es gestionen tant les altes com les edicions:

2.1 Generar una ID auto-incremental amb map i Math.max

Per trobar la ID més alta existent, transformem l'array d'objectes en un array de només IDs amb map i calculem el valor màxim.

- **Sintaxi:** Math.max(...array.map(element => element.propietat))
- **Exemple de sintaxi:**

```
const maxId = llista.length > 0 ? Math.max(...llista.map(x => x.id)) : 0;
```

2.2 Afegir un nou element amb push

Afegeix un element al final de l'array original (modifica l'array existent).

- **Sintaxi:** array.push(nouElement)

2.3 Modificar de manera immutable amb with (ES2023)

Si s'està editant un element, en lloc de modificar l'array directament per l'índex (array[i] = ...), el mètode with() retorna una **nova còpia de l'array** canviant només la posició indicada.

- **Sintaxi:** const nouArray = array.with(index, nouValor)
- **Exemple de sintaxi:**

```
const llistaActualitzada = llista.with(2, "Nou Valor");
```

Tasca 3: Eliminar de Manera Immutable (toSpliced)

A la funció eliminarProducte(id):

3.1 Cerca de l'índex amb findIndex

Retorna la posició del primer element que compleix la condició (o -1 si no existeix).

- **Sintaxi:** const index = array.findIndex(element => condicio)

3.2 Eliminar sense modificar l'original amb toSpliced (ES2023)

A diferència de splice (que modifica l'array original), toSpliced retorna un **nou array** sense l'element eliminat.

- **Sintaxi:** const nouArray = array.toSpliced(indexInici, quantitatAEliminar)
- **Exemple de sintaxi:**

```
const llistaSenseElement = llista.toSpliced(index, 1);
```

Tasca 4: Cerca Avançada del Darrer Element (findLast)

Al botó de cerca per categoria:

4.1 Cercar des del final amb findLast

Troba el **darrer element** de l'array que compleix una condició (recorre l'array de dreta a esquerra).

- **Sintaxi:** array.findLast(element => condicio)
- **Exemple de sintaxi:**

```
const ultimParell = nombres.findLast(n => n % 2 === 0);
```

Tasca 5: Ordenació per Columnes (toSorted i toReversed)

En clicar sobre les capçaleres de la taula:

5.1 Ordenar sense modificar l'original amb toSorted (ES2023)

A diferència de sort(), toSorted() retorna una còpia ordenada de l'array sense alterar l'ordre original.

- **Sintaxi:** const ordenat = array.toSorted((a, b) => comparacio)
- **Exemple de sintaxi:**

```
// Per a números:
const perPreu = productes.toSorted((a, b) => a.preu - b.preu);
// Per a cadenes de text:
const perNom = productes.toSorted((a, b) => a.nom.localeCompare(b.nom));
```

5.2 Invertir l'ordre amb toReversed (ES2023)

Retorna una nova còpia de l'array amb els elements en ordre invers.

- **Sintaxi:** const invertit = array.toReversed()

Tasca 6: Cerca i Filtres Combinats (filter i includes)

A la funció aplicarFiltres():

6.1 Filtrar per text i categoria amb filter i includes

El mètode filter retorna un subconjunt d'elements, i includes comprova si una cadena conté un text determinat.

- **Sintaxi:** array.filter(element => condicio)
- **Sintaxi includes:** cadena.includes(textCercat)
- **Exemple de sintaxi:**

```
const resultat = llista.filter(item =>
  item.nom.toLowerCase().includes(text.toLowerCase())
);
```

Resum de la Taula de Mètodes

Mètode	Categoria	Retorna	Modifica l'original?
push()	Mutació bàsica	Nova longitud	**Sí**
findIndex()	Cerca	Índex (number)	No
includes()	Cerca de text/array	Booleà (true/false)	No
at()	Accés	Element	No
findLast()	Cerca avançada	Element / undefined	No
map()	Transformació	Nou Array	No
filter()	Filtrat	Nou Array	No
reduce()	Acumulació	Un sol valor	No
some() / every()	Comprovació	Booleà (true/false)	No
with() **(ES2023)**	Immutabilitat	Nou Array	No
toSpliced() **(ES2023)**	Immutabilitat	Nou Array	No
toSorted() **(ES2023)**	Immutabilitat	Nou Array	No
toReversed() **(ES2023)**	Immutabilitat	Nou Array	No

Pregunta 1: Diferència entre `splice()` i `toSpliced()`

Pregunta: Quin és el principal benefici d'utilitzar `toSpliced()` (introduït a ES2023) en lloc del tradicional `splice()` quan treballem amb estats d'aplicacions o frameworks moderns?

- **Resposta esperada:** `splice()` és un mètode mutant que modifica directament l'array original sobre el qual s'executa. En canvi, `toSpliced()` és un mètode immutable: deixa l'array original intacte i en retorna una còpia modificada. En programació moderna (i en gestió d'estat), mantenir la immutabilitat evita efectes secundaris (*side effects*) no desitjats.

Pregunta 2: Comportament de `some()` vs `every()`

Pregunta: Quin serà el resultat de les següents dues línies d'execució si tenim un array de productes on un d'ells té preu `0` i la resta tenen preus positius?

JavaScript
```
const res1 = productes.some(p => p.preu === 0);
const res2 = productes.every(p => p.preu > 0);
```

- **A)** `res1 = true`, `res2 = true`
- **B)** `res1 = false`, `res2 = true`
- **C)** `res1 = true`, `res2 = false`
- **D)** `res1 = false`, `res2 = false`

- **Explicació:** `some()` retorna `true` perquè **almenys un** element compleix la condició (`preu === 0`). `every()` retorna `false` perquè **no tots** els elements compleixen la condició (`preu > 0`).

Pregunta 3: Rendiment de cerca amb `findLast()`

Pregunta: Si tenim una llista de 10.000 registres ordenats cronològicament (d'antics a nous) i volem obtenir la transacció més recent d'un usuari concret, per què és més eficient utilitzar `findLast()` que `filter()` o `find()`?

- **Resposta esperada:**
  1. Respecte a `filter()`: `filter()` ha de recórrer **tots** els 10.000 elements per construir un nou array, mentre que `findLast()` s'atura tan bon punt troba la primera coincidència.
  2. Respecte a `find()`: `find()` comença a cercar per l'inici (registres més antics). En canvi, `findLast()` comença a cercar des del final (registres més recents), per la qual cosa trobarà la transacció més recent molt més ràpidament sense haver de recórrer tota la llista antiga.

Pregunta 4: Mètodes immutables d'ES2023

Pregunta: Volem canviar el valor del tercer element d'un array de forma **immutable** (sense modificar l'array original). Quina de les següents opcions és la correcta utilitzant les funcions d'ES2023?

- **A)** `array[2] = "Nou Valor";`
- **B)** `const nouArray = array.with(2, "Nou Valor");`
- **C)** `const nouArray = array.splice(2, 1, "Nou Valor");`
- **D)** `const nouArray = array.at(2, "Nou Valor");`

- **Explicació:** `with(index, valor)` és el mètode dissenyat específicament a ES2023 per retornar una còpia d'un array actualitzant un element en una posició determinada de manera immutable.

Pregunta 5: Valor inicial en el mètode `reduce()`

Pregunta: Què passa si executem el mètode `reduce()` per calcular el valor total d'un inventari sense passar-li el segon argument (el valor inicial `0`), com en aquest codi?

```
const total = productes.reduce((acc, p) => acc + (p.preu * p.stock));
```

- **Resposta esperada:** El codi fallarà o retornarà un resultat incorrecte (`NaN`). Si no es proporciona un valor inicial a `reduce()`, el primer element de l'array s'assigna com a `acc` inicial (que és un objecte enter `{ id: 1, preu: ... }`) i la iteració comença des del segon element. En intentar sumar un objecte amb un número (`objecte + number`), la dada es corromp. Per tant, quan s'acumulen propietats d'un array d'objectes, el valor inicial (ex: `0`) és obligatori.